

ΙΟΝΙΟ ΠΑΝΕΠΙΣΤΗΜΙΟ
ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ



**Παραγοντοποίηση LU ενός Τετραγωνικού
πίνακα $N \times N$ με cache blocking, OpenMp
for constructs και OpenMp tasks**

Mohammad Matin Marzie, inf2022001
Διδάκνων: Μιχαήλ Στεφανιδάκη

Παράλληλος Προγραμματισμός 2026

Εισαγωγή: παραγοντοποίηση LU ενός τετραγωνικού πίνακα NxN από doubles (LU decomposition/factorization).....	2
Ορισμός	3
1. Σειριακή Επίλυση με cache blocking.....	3
(E1)	3
(E2)	3
2. Παράλληλη υλοποίηση αξιοποιώντας (1) και OpenMp for constructs	3
(E3)(E4)	3
3. Παράλληλοι μορφή του (1) χρησιμοποιώντας OpenMp tasks	3
(E5)(E6)	3
4. Σύγκριση απόδοσης και Συμπεράσματα	3
(E7)	3
5. Αναφορά στην Τεχνητή Νοημοσύνη	3
(E8)	3

Εισαγωγή: παραγοντοποίηση LU ενός τετραγωνικού πίνακα NxN από doubles (LU decomposition/factorization)

Ορισμός

Δοθέντος ενός τετραγωνικού πίνακα A, **άνω-κάτω αποσύνθεση** (lower-upper LU decomposition) ή **παραγοντοποίηση** (factorization) είναι η προσπάθεια να βρούμε δυο πίνακες L και U έτσι ώστε το γινόμενό τους να μας δώσει τον αρχικό πίνακα A, όπως φαίνεται και στον παρακάτω εικόνα [1]:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} \ell_{11} & 0 & 0 \\ \ell_{21} & \ell_{22} & 0 \\ \ell_{31} & \ell_{32} & \ell_{33} \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}.$$

$$A = L \times U \quad \text{ή} \quad [A] = [L] [U]$$

Κάθε τετραγωνικός πίνακας δεν μπορεί να αποσυνδεθεί στην παραπάνω μορφή. Για να είναι εφικτή αυτή η αποσύνθεση, προϋποθέτει ότι όλες οι κύριες ελλάσσονες [ορίζουσες](#) (leading principal minors) D_i του πίνακα A είναι διάφορες του μηδενός 0.

Μέθοδος του Doolittle [2]

Στην παρούσα εργασία επικεντρωνόμαστε στη **μέθοδο του Doolittle**, η οποία αποτελεί μια υπολογιστικά αποδοτική παραλλαγή της απαλοιφής Gauss ([Gaussian Elimination](#)). Το κύριο χαρακτηριστικό της είναι ότι επιβάλλει σταθερά την ύπαρξη μονάδων 1 στην κύρια διαγώνιο του κάτω τριγωνικού πίνακα L, επιτρέποντας στον άνω τριγωνικό πίνακα U να λάβει οποιεσδήποτε τιμές προκύψουν από τη διαδικασία της απαλοιφής.

Ο Κώδικας του [lu-serial.c](#) είναι μια υλοποίηση της μεθόδου Doolittle. Επιπλέον, για τους πίνακες L και U χρησιμοποιείται μόνο ένας πίνακας a για την αξιοποίηση της χωρικής τοπικότητας (spacial locality).

1. Σειριακή Επίλυση με cache blocking

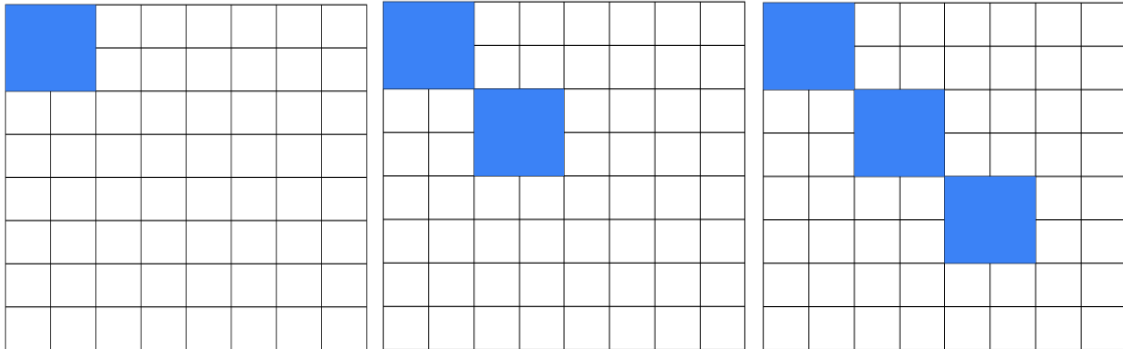
Για την περαιτέρω βελτίωση της χωρικής τοπικότητας στην προσπέλαση δεδομένων στις κρυφές μνήμες, εφαρμόζουμε την τεχνική cache blocking με μέγεθος κάθε block/tile να είναι bs.

(Ε.1)

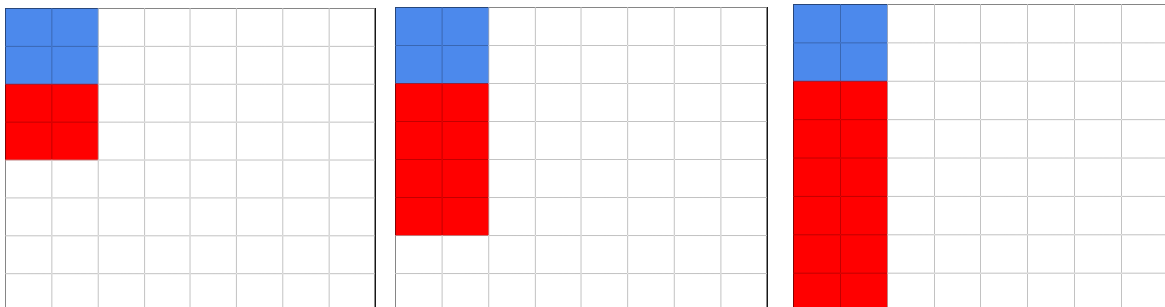
Έστω ότι έχουμε έναν πίνακα A διάστασης 8×8 με μέγεθος κάθε block $bs=2$. Η επεξεργασία των στοιχείων δεν γίνεται πλέον σειριακά ανά γραμμή, εκτελείται block ανά block. Έχουμε τον εξωτερικό βρόχο που διασχίζει τον πίνακα ανά block δείχνοντας πάντα στο πρώτο στοιχείο του κύριου διαγώνιου κάθε φορά.

```
for (int md = 0; md < N; md += bs) {}
```

Παρακάτω βλέπετε με μπλε χρώμα τα μπλοκ του κύριου διαγώνιου:



Στον καινούργιο αλγόριθμο λοιπόν, στο 1ο βήμα md , επεξεργάζονται τα στοιχεία του μπλοκ του κύριου διαγώνιου με μπλέ χρώμα, έπειτα στο ίδιο βήμα md υπολογίζονται τα στοιχεία των block που βρίσκονται στην ίδια στήλη (2L blocks) με το block του κύριου διαγώνιου, όπως φαίνονται με κόκκινο χρώμα:



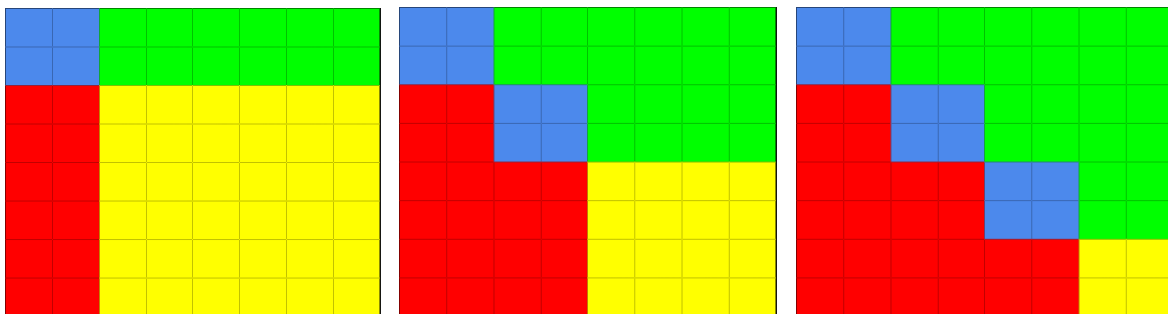
Κατόπιν υπολογισμού των 2L blocks, υπολογίζονται τα block που βρίσκονται στην ίδια γραμμή με την κύρια διαγώνιο (2U blocks), όπως απεικονίζονται με πράσινο χρώμα του ΠΑΣΟΚ:



Τέλος, πριν ολοκληρωθεί το 1ο βήμα της διαδικασίας (για το τρέχον md), ο αλγόριθμος προχωρά στην ενημέρωση των υπόλοιπων υποπινάκων του συστήματος, οι οποίοι ορίζονται στον κώδικα ως (3 Rest):



Στο 2ο βήμα, αυξάνεται η μεταβλητή md κατά bs και η ίδια διαδικασία επαναλαμβάνεται επαναληπτικά για το επόμενο επίπεδο:λ



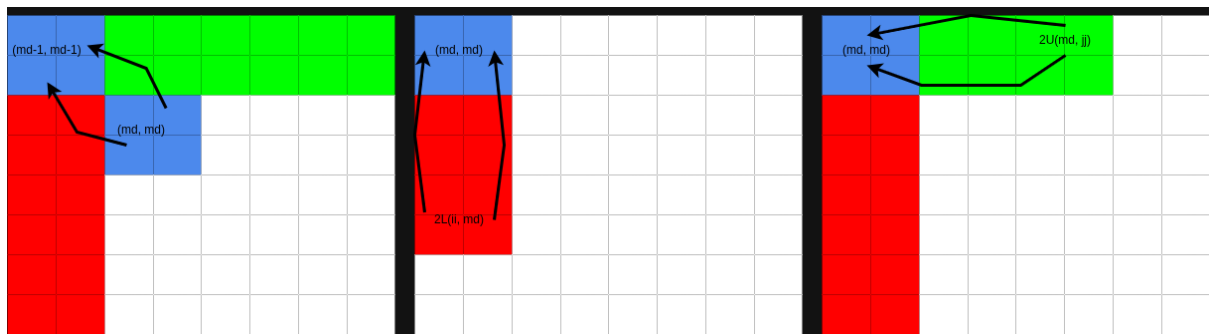
(E.2)

Λόγω των αλληλεξαρτήσεων των δεδομένων, ο υπολογισμός των block/tiles πρέπει να ακολουθεί συγκεκριμένη σειρά, όπως παρουσιάστηκε στην υποενότητα (E.1)

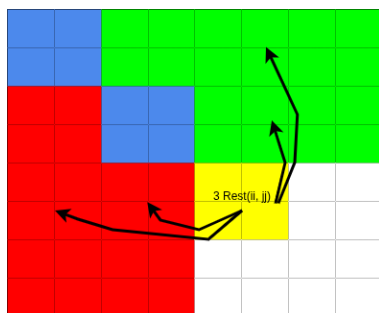
Πίνακας αλληλεξαρτήσεων

Όνομα block/tile	Εξαρτάται από τα block/tile
Diag(md, md)	Diag($md-bs, md-bs$)
2L(ii, md)	Diag(md, md)
2U(md, jj)	Diag(md, md)
3 Rest(ii, jj)	2L(ii, md) και 2U(md, jj)

Για κάθε md , τα block/tiles υπολογίζονται σε τέσσερα στάδια: πρώτα το διαγώνιο block Diag(md, md) που εξαρτάται ακριβώς από το προηγούμενο Diag($md-1, md-1$)



Στη συνέχεια τα block 2L και 2U παράλληλα μεταξύ τους (καθώς εξαρτώνται μόνο από το $\text{Diag}(md, md)$), και τέλος τα υπόλοιπα 3 Rest block $\text{Rest}(ii, jj)$ που απαιτούν την ολοκλήρωση και των δύο. Η διαδικασία επαναλαμβάνεται για κάθε επόμενο md .



2. Παράλληλη υλοποίηση αξιοποιώντας (1) και OpenMp for constructs

(E.3) Λογική και τήρηση αλληλεξαρτήσεων

Η παράλληλη περιοχή (`#pragma omp parallel { }`) περικλείει ολόκληρο τον εξωτερικό βρόχο md , ώστε τα νήματα να δημιουργούνται μία φορά και να επαναχρησιμοποιούνται σε κάθε επανάληψη.

Οι αλληλεξαρτήσεις που περιγράφηκαν στο (E.2) τηρούνται μέσω των **implicit barriers** που παρέχουν οι OpenMP οδηγίες, ως εξής:

Βήμα 1 — Διαγώνιο block (`#pragma omp single`): Το διαγώνιο block $\text{Diag}(md, md)$ πρέπει να υπολογιστεί πλήρως πριν ξεκινήσει οποιοδήποτε άλλο βήμα, καθώς όλα εξαρτώνται από αυτό. Η οδηγία `omp single` εξασφαλίζει ότι μόνο ένα νήμα εκτελεί τον υπολογισμό, ενώ το **implicit barrier** στο τέλος του `single` αναγκάζει όλα τα υπόλοιπα νήματα να περιμένουν μέχρι την ολοκλήρωσή του.

Βήμα 2L — L-blocks (`#pragma omp for schedule(dynamic) nowait`): Τα blocks κάτω από το διαγώνιο εξαρτώνται μόνο από το $\text{Diag}(md, md)$, το οποίο έχει ήδη ολοκληρωθεί. Κάθε block $2L(ii, md)$ είναι ανεξάρτητο από τα άλλα 2L blocks, οπότε μοιράζονται

δυναμικά στα νήματα. Το `nowait` επιτρέπει στα νήματα που τελειώνουν νωρίς να προχωρήσουν αμέσως στο βήμα 2U, χωρίς να περιμένουν τα υπόλοιπα.

Βήμα 2U — U-blocks (#pragma omp for schedule(dynamic)): Τα blocks δεξιά του διαγωνίου εξαρτώνται επίσης μόνο από το `Diag(md,md)` και είναι ανεξάρτητα μεταξύ τους, οπότε παραλληλίζονται ομοίως. Εδώ **δεν** χρησιμοποιείται `nowait`, οπότε υπάρχει `implicit barrier` στο τέλος: όλα τα νήματα περιμένουν μέχρι να ολοκληρωθούν και τα τελευταία 2U blocks, πριν προχωρήσουν στο βήμα 3. Αυτό είναι απαραίτητο γιατί τα 3 Rest blocks εξαρτώνται και από τα 2L **και** από τα 2U.

Βήμα 3 — Trailing submatrix (#pragma omp for collapse(2) schedule(dynamic)): Τα υπόλοιπα blocks ενημερώνονται παράλληλα αφού έχουν ολοκληρωθεί τα 2L και 2U. Το `implicit barrier` στο τέλος εξασφαλίζει ότι η επόμενη επανάληψη `md` δεν ξεκινά πριν ολοκληρωθούν όλες οι ενημερώσεις.

(E.4)

Η οδηγία **`collapse(2)`** [3] χρησιμοποιήθηκε στο βήμα 3 (trailing submatrix) δεν διδάχθηκε στο μάθημα.

```
#pragma omp for collapse(2) schedule(dynamic)
for (int ii = md_end; ii < N; ii += bs) {
    for (int jj = md_end; jj < N; jj += bs) {
        ...
    }
}
```

Με το `collapse(2)` οι **δύο βρόχοι συγχωνεύονται** σε έναν ενιαίο χώρο επαναλήψεων μεγέθους $((N-md)/bs)^2$, αυξάνοντας σημαντικά τον βαθμό παραλληλισμού, ιδίως στα τελευταία βήματα `md` όπου ο εξωτερικός βρόχος έχει λίγες επαναλήψεις. Αυτό σε συνδυασμό με το `schedule(dynamic)` εξασφαλίζει καλύτερη κατανομή φόρτου μεταξύ των νημάτων.

3. Παράλληλοι μορφή του (1) χρησιμοποιώντας OpenMp tasks

(E.5)

Η σωστή σειρά υπολογισμού εξασφαλίζεται μέσω των `depend clauses` σε κάθε task:

Diagonal block (md,md): `depend(inout: a[md*N+md])`

- inout: διαβάζει και γράφει το ίδιο block. Επόμενα tasks που αγγίζουν αυτό το στοιχείο θα περιμένουν.

2L blocks (ii, md): depend(in: a[md*N+md]) depend(out: a[ii*N+md])

- in: a[md*N+md]: περιμένει να τελειώσει το diagonal task
- out: a[ii*N+md]: δηλώνει ότι γράφει το block (ii,md), ώστε τα block-3 tasks να περιμένουν

2U blocks (md, jj): depend(in: a[md*N+md]) depend(out: a[md*N+jj])

- in: a[md*N+md]: περιμένει το diagonal
- out: a[md*N+jj]: δηλώνει εγγραφή στο (md,jj) για τα block-3 tasks

Block-3 (ii, jj): depend(in: a[ii*N+md], a[md*N+jj]) depend(out: a[ii*N+jj])

- in: a[ii*N+md]: περιμένει το αντίστοιχο 2L task
- in: a[md*N+jj]: περιμένει το αντίστοιχο 2U task
- out: a[ii*N+jj]: δηλώνει εγγραφή για την επόμενη iteration του md

(E.6)

	Βήμα 2 (OpenMp for)	Βήμα 3 (OpenMp tasks)
Μηχανισμός	Έμμεσο barriers μεταξύ φάσεων	depend() clauses ανά task
Granularity	Φάση-φάση (1 → 2 → 3)	Block-block (fine-grained)
Αναμονή	Όλα τα threads περιμένουν στο barrier	Μόνο τα εξαρτημένα tasks περιμένουν

Γιατί το task scheme έχει μεγαλύτερο παραλληλισμό:

Στο omp for, το implicit barrier μετά τα 2U blocks αναγκάζει **όλα τα threads** να **περιμένουν** πριν ξεκινήσουν τα block-3, ακόμα και αν κάποια block-3 tasks έχουν ήδη τα δεδομένα που χρειάζονται.

Στο task scheme, ένα block-3 task (ii, jj) μπορεί να ξεκινήσει μόλις τελειώσουν τα συγκεκριμένα 2L (ii,md) και 2U (md,jj) tasks, ανεξάρτητα από τα υπόλοιπα. Έτσι **tasks** από **διαφορετικές φάσεις** μπορούν να **τρέχουν ταυτόχρονα**.

4. Σύγκριση απόδοσης και Συμπεράσματα

(Ε.7)

```
=== TILESIZE = 16 ===

-- N = 512 --
serial          best=0.0651s  runs=[0.0651]
serial-blocked  best=0.0239s  runs=[0.0239]
omp-for-blocked best=0.0146s  runs=[0.0146]
omp-tasks-blocked best=0.0217s  runs=[0.0217]

-- N = 1024 --
serial          best=0.7961s  runs=[0.7961]
serial-blocked  best=0.1984s  runs=[0.1984]
omp-for-blocked best=0.0812s  runs=[0.0812]
omp-tasks-blocked best=0.1206s  runs=[0.1206]

-- N = 2048 --
serial          best=23.1772s  runs=[23.1772]
serial-blocked  best=1.6618s  runs=[1.6618]
omp-for-blocked best=0.6702s  runs=[0.6702]
omp-tasks-blocked best=1.0235s  runs=[1.0235]

-- N = 2496 --
serial          best=15.7976s  runs=[15.7976]
serial-blocked  best=2.3571s  runs=[2.3571]
omp-for-blocked best=0.8698s  runs=[0.8698]
omp-tasks-blocked best=1.8536s  runs=[1.8536]
```

1. Serial vs Serial-blocked:

Το cache-blocked serial είναι **3-8x γρηγορότερο** από το απλό serial. Αυτό οφείλεται αποκλειστικά στην καλύτερη χρήση της cache — ο αλγόριθμος είναι ο ίδιος, απλά τα δεδομένα φορτώνονται πιο αποδοτικά.

2. Serial-blocked vs Parallel:

Τα parallel versions δίνουν επιπλέον **3-4x speedup** πάνω από το serial-blocked για μεγάλα N, αξιοποιώντας τους πολλαπλούς πυρήνες.

3. omp-for vs omp-tasks:

Για **μικρά N** (512): παρόμοια απόδοση, το task overhead δεν φαίνεται

Για **μεγάλα N** (2048, 2496): το omp-for είναι **ελαφρώς καλύτερο** στα περισσότερα cases, παρά τη **θεωρητική υπεροχή** του task scheme — το **overhead δημιουργίας tasks** μετράει πρακτικά

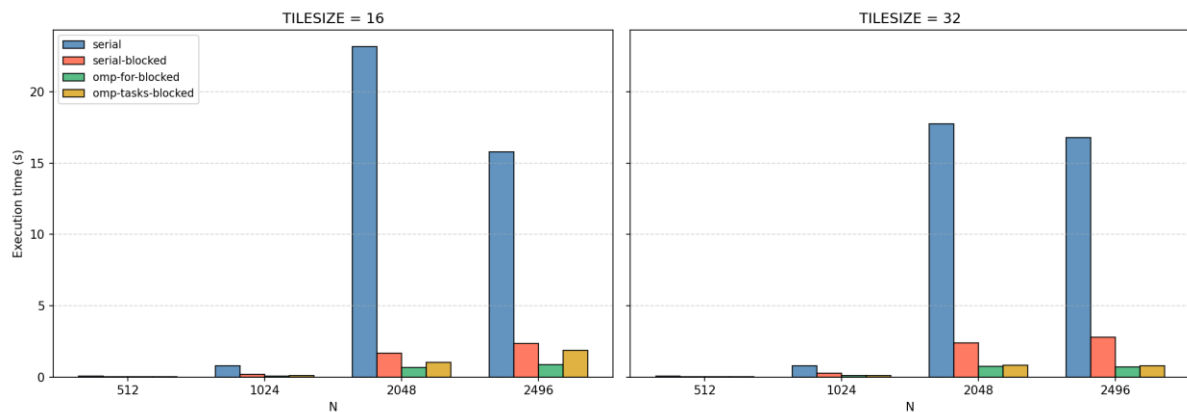
```
=== TILESIZE = 32 ===

-- N = 512 --
serial          best=0.0636s  runs=[0.0636]
serial-blocked  best=0.0357s  runs=[0.0357]
omp-for-blocked best=0.0121s  runs=[0.0121]
omp-tasks-blocked best=0.0123s  runs=[0.0123]

-- N = 1024 --
serial          best=0.7844s  runs=[0.7844]
serial-blocked  best=0.2590s  runs=[0.2590]
omp-for-blocked best=0.0900s  runs=[0.0900]
omp-tasks-blocked best=0.1234s  runs=[0.1234]

-- N = 2048 --
serial          best=17.7529s  runs=[17.7529]
serial-blocked  best=2.4077s  runs=[2.4077]
omp-for-blocked best=0.7509s  runs=[0.7509]
omp-tasks-blocked best=0.8217s  runs=[0.8217]

-- N = 2496 --
serial          best=16.8093s  runs=[16.8093]
serial-blocked  best=2.7909s  runs=[2.7909]
omp-for-blocked best=0.6983s  runs=[0.6983]
omp-tasks-blocked best=0.7920s  runs=[0.7920]
```



Το BLOCKSIZE/TILESIZE=16 δίνει block/tiles των $16 \times 16 \times 8$ bytes = 2KB που **χωράνε άνετα στην L1 cache**. Το TILESIZE=32 δίνει $32 \times 32 \times 8$ = 8KB που μπορεί να προκαλεί περισσότερα **cache misses στην L1** αλλά λιγότερο task/scheduling overhead λόγω λιγότερων tiles.

Ανομαλία: N=24: N=2496 serial πιο γρήγορο από N=2048 serial:

Παρατηρούμε ότι το serial για N=2496 (9.69s) είναι γρηγορότερο από N=2048 (12.47s) με TILESIZE=16, κάτι που φαίνεται παράδοξο.

5. Αναφορά στην Τεχνητή Νοημοσύνη

(Ε.8)

Η χρήση της Τεχνητής Νοημοσύνης διαδραμάτισε επικουρικό ρόλο καθ' όλη την διάρκεια της εκπόνησης της παρούσας εργασίας. Συγκεκριμένα: στην εύρεση εξαρτήσεων μεταξύ των block, έλεγχος της ορθότητας του κώδικα, εύρεση οδηγιών collapse(2) όπως αναφέρθηκε στο (Ε.4) και depend(inout) κατά την υλοποίηση της (3) αλλά και στην δημιουργία διαγραμμάτων των (Ε.7). Η ΤΝ έχει βοηθήσει επίσης σε μερικά σημεία της σύνταξης της αναφοράς.

Βιβλιογραφία

- [1] Wikipedia contributors, "LU decomposition," *Wikipedia, The Free Encyclopedia*, [\[link\]](#) (τελευταία πρόσβαση 02/06/2026).
- [2] GeeksforGeeks, [\[link\]](#) (τελευταία πρόσβαση 02/06/2026)
- [3] OpenMp.org [\[link\]](#) (τελευταία πρόσβαση 04/06/2026)